

AI Terimleri ve Personal RAG Agent Projesindeki Karsiliklari

Bu dokuman, projede aktif olarak gecen AI ve backend terimlerini aciklar. Her terim icin ne oldugu, neden onemli oldugu, projede nasil kullanildigi ve kısa bir ornek verilir.

Katman	Bu projede bugunku durum
Client	Telegram bot ilk arayuz olarak kullaniliyor.
.NET Backend	Orchestration, persistence, formatting ve search sorgu akisini yönetiyor.
Python AI Service	Intent, extraction ve summary kararlarini uretiyor.
Database	PostgreSQL kayitlari, ozetleri ve content kind bilgilerini sakliyor.
Gelecek adım	Embedding, pgvector, retrieval, RAG ve agent workflow.

Cekirdek Terimler

Asagidaki terimler bugunku uygulamada aktif olarak karřina cikan veya hemen sonraki fazlarda bilmen gereken kavramlardir.

LLM - Large Language Model

- Tanım: Büyük miktarda metin üstünde eğitilmiş ve yeni metin üretebilen modeldir. Bizim projede Gemini modeli hem intent anlamada hem özet üretmede kullanılır.
- Neden önemli: Kullanıcının niyetini anlamak, özet çıkarmak ve daha sonra RAG cevabı üretmek için temel akıldır.
- Projede kullanımı: Python AI service içinde intent provider ve summary provider katmanında kullanılıyor.
- Mini örnek: Kullanıcı 'bugün attığım videoları listele' yazınca model bunun bir search isteği olduğunu çıkarıyor.

```
IntentResponse -> intent: search, content_kind: video
```

Prompt

- Tanım: Modele verdiğimiz talimat metnidir. Ne yapmasını, hangi dilde cevap vermesini ve hangi formatta dönmesini prompt belirler.
- Neden önemli: Aynı model kötü prompt ile zayıf, iyi prompt ile çok daha tutarlı sonuç verir.
- Projede kullanımı: Summary prompt ile modele 'Türkçe özet çıkar ve JSON şemasına uy' diyoruz.
- Mini örnek: Summary akışında modelden başlık, kısa özet ve ana noktalar isteniyor.

```
content_summary_v1.txt -> always output Turkish summary fields
```

Structured Output

- Tanım: Modelin serbest paragraf yerine belirli alanlara sahip düzenli bir çıktı üretmesidir. Genelde JSON olarak döner.
- Neden önemli: Backend tarafında parse etmek kolaylaşır. Başlık, özet, etiket gibi alanları güvenli şekilde kullanırsın.
- Projede kullanımı: Summary endpoint title, short_summary, key_points, tags, language, provider alanları döndürüyor.
- Mini örnek: Telegram mesajına serbest uzun cevap yerine düzenli kart biçimi basabiliyoruz.

```
{"title":"RAG nedir?","short_summary":"...","key_points":["..."]}
```

JSON Schema

- Tanım: Modelin veya API'nin hangi alanları döndüreceğini tarif eden sözleşmedir.
- Neden önemli: Hem model çıktısını hem servisler arası kontratı daha güvenilir yapar.
- Projede kullanımı: Gemini summary ve intent çağrılarında beklenen alanlar şema ile tanımlanıyor.
- Mini örnek: intent must be save, search or clarify gibi kısıtlar schema mantığıyla uygulanıyor.

```
required: ["intent", "content_kind", "source_type", "time_filter", "keywords"]
```

Tokens ve Context Window

- Tanım: Token modelin işlediği küçük metin birimidir. Context window modelin tek seferde okuyabildiği toplam sınırdır.
- Neden önemli: Uzun PDF, makale veya transcript geldiğinde hepsini tek seferde veremezsin. Bu yüzden parçalama gerekir.
- Projede kullanımı: Şu an kısa içerikler için doğrudan summary yapıyoruz; ileride uzun dokümanlarda chunking ve retrieval gerekecek.
- Mini örnek: 50 sayfalık PDF geldiğinde tüm metni tek çağrıda göndermek yerine parçalara ayıracağız.

```
Bugün: short text -> summary | Gelecek: chunks -> retrieval -> answer
```

Intent Classification

- Tanım: Kullanıcının mesajıyla ne yapmak istediğini anlamaktır. Save mi, search mü, açıklama mı istiyor bunu çıkarır.
- Neden önemli: Aynı sohbet kanalında hem kayıt hem arama hem de soru sorma davranışı olabilir. Akışı ayıran parça intent'tir.
- Projede kullanımı: Python /api/v1/intents endpoint'i intent, content_kind, source_type ve keywords döndürüyor.
- Mini örnek: Kullanıcı link attıysa save, 'bugün attığım videoları listele' dediyse search.

```
{"intent":"search","content_kind":"video","time_filter":"today","keywords":[]}
```

Extraction

- Tanım: Ham içeriği işlenebilir hale getirme adıdır. Linkten metin, başlık, metadata veya transcript çıkarılır.
- Neden önemli: LLM ham URL'yi doğrudan anlamaz. Önce içerik çıkarılmalı ki özet veya retrieval kaliteli olsun.
- Projede kullanımı: ArticleExtractor sayfadan metin alıyor, YouTubeExtractor metadata topluyor, ayrıca detected_content_kind dönüyor.
- Mini örnek: Dailymotion linkinde source_type article olabilir ama detected_content_kind video olabilir.

```
{"source_type":"article","detected_content_kind":"video","extracted_text":"..."}
```

SourceType ve ContentKind

- Tanım: SourceType içeriğin geldiği kaynağı anlatır. ContentKind ise içeriğin ne tür bir şey olduğunu anlatır.
- Neden önemli: Kaynak ile içerik tipi aynı şey değildir. Dailymotion article gibi kaydolsa bile aslında video olabilir.
- Projede kullanımı: source_type: youtube/article/pdf/image/telegram, content_kind: video/text/image olarak tutuluyor.
- Mini örnek: Instagram reel -> source_type instagram, content_kind video.

```
SourceType = nereden geldi | ContentKind = neyin kendisi
```

Terimler - Devam

Summarization

- Tanım: Çıkarılan içeriğin kısa, okunabilir ve yapısal özete dönüştürülmesidir.
- Neden önemli: Kullanıcının tüm makaleyi tekrar okumasına gerek kalmaz. Ayrıca arama sonuçlarını daha anlaşılır hale getirir.
- Projede kullanımı: Python /api/v1/summaries endpoint'i ve backend summary proxy akışı bunu yapıyor.
- Mini örnek: Telegram'da Baslik, Ozet, Ana noktalar şeklinde dönen cevap bu katmandan gelir.

```
SummaryResponse -> title, short_summary, key_points, tags
```

Validation ve 422 Hatası

- Tanım: API'ye gelen request'in kurallara uygun olup olmadığını kontrol etmektir. Uygun değilse 422 gibi hata döner.
- Neden önemli: Servisler arası kontratı sağlam tutar. Eksik alan, yanlış tip, kısa text gibi hataları erken yakalar.
- Projede kullanımı: FastAPI request modelleri pydantic ile doğrulanıyor.
- Mini örnek: Summary text alanı minimum uzunluğa uymadığında 422 aldık ve oradan sorunu bulduk.

```
text: str = Field(min_length=1, max_length=20000)
```

DTO - Data Transfer Object / Contract

- Tanım: Servisler arası taşınan veri modelleridir. Request ve response sınıfları gibi düşün.
- Neden önemli: Backend ile AI service birbirine serbest veri değil, sözleşmeli veri gönderir.
- Projede kullanımı: CreateExtractionRequest, CreateSummaryResponse, ClassifyIntentResponse gibi sınıflar var.
- Mini örnek: Bir alan adı değişirse iki servis de etkilenir. O yüzden contract yönetimi önemlidir.

```
CreateExtractionResponse(ContentId, SourceType, DetectedContentKind, ...)
```

Orchestration

- Tanım: Bir işlemin hangi sırayla, hangi servisler çağrılarak yapılacağını yönetmektir.
- Neden önemli: Tek başına model yeterli değildir. Önce extraction, sonra summary, sonra DB save gibi zincir gerekir.
- Projede kullanımı: .NET backend orchestrator gibi davranıyor. Telegram mesajını alıp uygun akışa sokuyor.
- Mini örnek: URL varsa extraction -> summary -> save. Search isteği varsa intent -> DB search -> format response.

```
Telegram -> Backend -> AI service -> DB -> Telegram
```

Keyword Search

- Tanım: Kelimelerin başlıkta, özetle veya ham metinde geçip geçmediğine göre arama yapmaktır.
- Neden önemli: Semantic search gelmeden önce temel arama davranışını sağlar.
- Projede kullanımı: EF Core repository title, short summary ve rawText alanlarında ILIKE ile arama yapıyor.
- Mini örnek: 'mantı tarifi' yazarsan başlık veya özet içinde bu kelimeler aranır.

```
EF.Functions.ILike(content.Summary.Title, "%mantı%")
```

Retrieval

- Tanım: Kullanıcı sorusuna en alakalı kayıtları bulup getirme adıdır.
- Neden önemli: RAG'in ilk yarısı retrieval'dır. Doğru içeriği bulamazsan iyi cevap da üretemezsin.
- Projede kullanımı: Şu an keyword search ile temel retrieval var; ileride embedding tabanlı retrieval gelecek.
- Mini örnek: 'dün attığım videoları listele' dediğinde tarih filtresi artı içerik türü filtresi çalışıyor.

```
SearchContentsQuery(keywords, contentKind, sourceType, fromUtc, toUtc)
```

Embedding

- Tanım: Metni anlam uzayında sayısal vektöre dönüştüren temsildir.
- Neden önemli: Kelime birebir geçmese de anlamca yakın içerikleri bulmak için gerekir.
- Projede kullanımı: Henüz aktif değil ama RAG aşamasında kayıtların embedding'i üretilecek.
- Mini örnek: 'mantı tarifi' ile 'kayseri mantısı nasıl yapılır' benzer bulunabilir.

```
text -> embedding vector -> similarity search
```

RAG - Retrieval Augmented Generation

- Tanım: Önce ilgili içerikleri bulup sonra modeli bu içeriklerle besleyerek cevap üretme yaklaşımıdır.
- Neden önemli: Model kendi hafızasından uydurmak yerine senin kayıtların üstünden cevap verir.
- Projede kullanımı: Şu an buna hazırlık aşamasındayız: extraction, summary, retrieval foundation kuruldu.
- Mini örnek: 'geçen hafta attığım RAG videolarının ortak noktası ne' sorusu ileride RAG ile cevaplanacak.

```
query -> retrieve relevant chunks -> give context to LLM -> answer
```

Tool Calling ve Agent

- Tanım: Modelin dış araçları çağırarak çok adımlı iş yapmasına tool calling denir. Bu akışın koordineli haline agent denir.
- Neden önemli: Sadece cevap vermek yerine arama yapıp sonuç toplayan ve sonra cevap üreten akışlar kurulur.
- Projede kullanımı: Henüz tam agent yok; ama intent, extraction, summary ve search araçları agent'a uygun biçimde ayrıştırıldı.
- Mini örnek: İleride 'dünkü fitness videolarını bul, özetlerini kıyasla, en iyi üçünü öner' gibi talepler agentik olur.

Reason -> Search tool -> Observe -> Summarize -> Reply

Bu Projede Akis Ozetleri

Kullanici link gonderir

Akis: Telegram -> .NET -> Python extraction -> Python summary -> PostgreSQL -> Telegram cevabi

Ne ogreniriz: Extraction, content kind detection, summary ve orchestration birlikte calisir.

Kullanici arama ister

Akis: Telegram -> .NET -> Python intent -> backend search query -> PostgreSQL -> Telegram liste cevabi

Ne ogreniriz: Intent classification ile retrieval'in ilk hali gorulur.

Gelecekte RAG sorusu

Akis: Query -> retrieval -> ilgili icerikler -> LLM answer generation

Ne ogreniriz: Keyword search yerine semantic retrieval devreye girer.

Calisma Notu

Bu projede sirayla su kaslari guclendiriyorsun: once backend orchestration, sonra AI service contracts, sonra retrieval, sonra RAG, en son agentik workflow. Yani bugun gordugun terimler rastgele degil; hepsi ileride semantic search ve personal knowledge assistant vizyonuna baglaniyor.